

**Спонсор:** AWS — переход от SaaS к агентному SaaS с ресурсами для независимых поставщиков программного обеспечения на всех уровнях стека.

[Узнайте, как искусственный интеллект для независимых поставщиков программного обеспечения превращает визуальные данные в результаты](#)

## МСП без сохранения состояния вновь пробудил мой интерес (и вдохновил на создание mcp-explorer и datasette-mcp)

31 июля 2026 года

Вторник был [днём МСП без сохранения состояния](#) — днём выхода MCP 2.0, или [спецификации протокола Model Context от 28 июля 2026 года](#), если использовать более формальное, но менее запоминающееся название. Это самое значительное изменение в спецификации МСП с момента ее появления, которое также возродило мой личный интерес к протоколу.

Для справки: МСП — это протокол Model Context Protocol, который описывает стандартный способ предоставления новых инструментов для агентных фреймворков на основе больших языковых моделей. Он был представлен компанией Anthropic [в ноябре 2024 года](#), вызвал огромный всплеск интереса в 2025 году, а затем его несколько затмили [Skills](#) (еще одно изобретение Anthropic), когда стало очевидно, что агентский интерфейс с доступом к терминалу и curl может выполнять большинство функций МСП более гибким способом. Я писал об этом [в своем обзоре за 2025 год](#).

Теперь я возвращаюсь к МСП. Предоставление агенту среды оболочки с доступом к интернету [чревато рисками](#) и требует мощной модели, способной эффективно управлять такой средой. Инструменты МСП проще поддаются аудиту и контролю, и они достаточно просты, чтобы их могли использовать небольшие модели, работающие на ноутбуке.

Новая спецификация МСП без сохранения состояния также значительно упрощает реализацию как клиентов, так и серверов для этого протокола. На этой неделе я создал три таких сервера!

### Что проще с МСП без отслеживания состояния #

Лучше всего разницу между МСП с отслеживанием состояния и без него можно увидеть в этом [посте в блоге от 21 мая](#), в котором представлен RC новой спецификации. В нем приведен наглядный пример «до и после».

Старый МСП с отслеживанием состояния (я буду называть его «устаревшим МСП») требовал двух HTTP-запросов: первый — для инициализации сеанса и получения MCP-Session-Id, а второй — для фактического вызова инструмента:

```
POST /mcp HTTP/1.1
```

```
Content-Type: application/json
```

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-11-25",
    "capabilities": {
    },
    "clientInfo": {
      "name": "my-app",
      "version": "1.0"
    }
  }
}
```

```
POST /mcp HTTP/1.1
```

```
Mcp-Session-Id: 1868a90c-3a3f-4f5b
```

```
Content-Type: application/json
```

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "search",
    "arguments": {
      "q": "otters"
    }
  }
}
```

Новый способ без отслеживания состояния использует один HTTP-запрос, который выглядит так:

```
POST /mcp HTTP/1.1
MCP-Protocol-Version: 2026-07-28
Mcp-Method: tools/call
Mcp-Name: search
Content-Type: application/json
```

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "search",
    "arguments": {
      "q": "otters"
    }
  },
  "_meta": {
    "io.modelcontextprotocol/clientInfo": {
      "name": "my-app",
      "version": "1.0"
    }
  }
}
```

Это намного удобнее как с точки зрения реализации на стороне клиента, так и на стороне сервера. Кроме того, такой подход лучше подходит для создания масштабируемых веб-приложений, поскольку теперь вам не нужно поддерживать состояние на стороне сервера для отслеживания идентификаторов сеансов или беспокоиться о том, чтобы один и тот же сеанс направлялся на один и тот же сервер.

## mcp-explorer #

Я не смог найти хороший инструмент командной строки для интерактивного тестирования MCP-сервера, поэтому попросил Codex помочь мне создать свой собственный.

[mcp-explorer](#) — вот что у меня получилось. Это инструмент командной строки Python без сохранения состояния, поэтому вам даже не нужно его устанавливать, чтобы опробовать — он работает с [uvx](#) следующим образом:

```
uvx mcp-explorer list https://agentic-mermaid.dev/mcp
```

Эта команда запрашивает демонстрационный MCP [agentic-mermaid.dev](#) от Аде Ошинье. Приведенная выше команда возвращает следующий список инструментов:

```
execute(code: string, timeoutMs?: integer) - Execute Mermaid SDK code
  Run JavaScript in an isolated sandbox; return a value.
```

```
describe_sdk(family: string, detail?: string) - Describe Mermaid SDK operations
  Return version-matched mutation operations for one diagram family.
```

```
render_svg(source: string, options?: object) - Render Mermaid as SVG
  Render a Mermaid source string to themeable SVG. Returns { ok, svg }.
```

```
render_ascii(source: string, useAscii?: boolean, targetWidth?: integer, options?:
object) - Render Mermaid as text
```

Render a Mermaid source string to text. Returns { ok, text }.

```
render_png(source: string, scale?: number, background?: string, fitTo?: object,  
options?: object) - Render Mermaid as PNG  
  Rasterize a Mermaid source string to PNG. Returns { ok, png_base64 }.  
...
```

Then to inspect a tool:

```
uvx mcp-explorer inspect render_svg
```

This outputs a whole bunch of information, including the JSON schema of the inputs and outputs.

To call that tool and pass arguments to it:

```
uvx mcp-explorer call \  
  https://agentic-mermaid.dev/mcp \  
  render_svg \  
  -a source 'graph TD; A-->B' \  
  -a options '{"padding":24}'
```

Which returns:

```
{"ok":true,"svg":"<svg xmlns=\"http://www.w3.org/2000/svg\" width=...
```

To get just the raw SVG try adding `| jq .svg -r` to that command. I got back [this image](#):



There are a [few more commands](#) in the README, but you get the general idea. I find building CLI tools like this to be a really productive way to get familiar with a specification, even if an agent writes most of the actual code.

## datasette-mcp #

The second project is [datasette-mcp](#), a Datasette plugin which adds a `/-/mcp` endpoint to any Datasette instance.

This is probably the fourth time I've tried building this plugin, but thanks to the new stateless MCP specification I finally have a version that feels good to release.

It provides just three tools: `list_databases()`, `get_database_schema(database_name)`, and `execute_sql(database_name, sql)`. They do exactly what you would expect them to do—though `execute_sql()` is read-only for the moment.

Wire these into an agent, or a chat tool like ChatGPT or Claude, and they'll gain the ability to run SQL queries against your hosted Datasette instance.

So far I'm running it on the Datasette mirror of my blog, at [datasette.simonwillison.net/-/mcp](#). It took a bit of fiddling to figure out how to attach that to ChatGPT and Claude, but I got there in the end. Here's [a new TIL](#) showing exactly how to do that.

Here's [a shared Claude session](#) where I asked it:

```
list tables in simonwillison.net
```

And then:

```
what has Simon said recently about MCP?
```

It ran 7 separate SQL queries to figure out the answer.

## llm-mcp-client #

My [LLM tool](#) is long overdue for an official MCP integration. The new alpha [llm-mcp-client](#) plugin is my attempt at exactly that:

```
llm install llm-mcp-client
llm -T 'MCP("https://datasette.simonwillison.net/-/mcp")' 'count the notes'
```

Here's the output (including reasoning trace, I'm using [LLM 0.32rc2](#)):

### Considering note count

*I see the question "count the notes" is probably asking me to tally up blog notes. It could also mean published notes or drafts, so there's some ambiguity there. I'll need to figure out the total number of notes, likely by querying the count for both published notes and drafts to get a clear answer. Let's execute that count!*

There are **151 notes**.

And [the output of llm logs](#) for that prompt.

Once this is fully baked, I'm considering bringing it directly into LLM core. I'm excited to experiment with MCP in [Datasette Agent](#) and [llm-coding-agent](#) as well.

## MCP is a safer way to build with agents #

A few months after MCP was first released, I wrote [Model Context Protocol has prompt injection security problems](#), where I noted that the pattern of having end users mix and match tools pushed responsibility for avoiding data exfiltration attacks out to the users themselves. I hadn't coined [the Lethal Trifecta](#) yet, but that was absolutely what I had in mind.

Then general agents with arbitrary shell and curl access came along, and that's so much harder to keep secure!

Something I've come to appreciate about MCP is that it's much easier to reason about agent capabilities and what might go wrong than with arbitrary command execution in an open network environment—the default for most of today's general and coding agent tools.

I plan to lean into MCP a whole lot more when I'm building sensitive applications on top of LLMs.

---

Posted [31st July 2026](#) at 11:13 pm · Follow me on [Mastodon](#), [Bluesky](#), [Twitter](#) or [subscribe to my newsletter](#)

### More recent articles

- [OpenAI's accidental cyberattack against Hugging Face is science fiction that happened](#) - 22nd July 2026
- [A Fireside Chat with Cat and Thariq from the Claude Code team](#) - 21st July 2026

projects 550

ai 2,161

datasette 1,531

mermaid 5

generative-ai 1,913

llms 1,880

llm 617

anthropic 321

model-context-protocol 31

Previous: [OpenAI's accidental cyberattack against Hugging Face is science fiction that happened](#)

## Monthly briefing

Sponsor me for **\$10/month** and get a curated email digest of the month's most important LLM developments.

Pay me to send you less!

[Sponsor & subscribe](#)